

cosa è il risc-v?

RISC-V È UNA ISA, UN INSIEME DI REGOLE E ISTRUZIONI CHE UN PROCESSORE DEVE CAPIRE. È UN LINGUAGGIO UNIVERSALE SVILUPPATO A PARTIRE DAGLI ANNI 2010. GLI OBIETTIVI SONO DI MASSIMIZZARE LE PRESTAZIONI, MINIMIZZARE I COSTI, RIDUCENDO I TEMPI DI SVILUPPO.

VERSIONI BASE :

- **RV32I** : 32 BIT $\rightarrow$ 40 INSTRUCTIONS, 32 REGISTERS.
- **RV32E** : 32 BIT $\rightarrow$ " " , 16 REGISTERS (E = **EMBEDDED**).
- **RV64I** : 64 BIT $\rightarrow$ " " , 32 REGISTERS.
- **RV128I** : 128 BIT $\rightarrow$ " " , 32 REGISTERS.

ESTENSIONI, CHE AUMENTANO I COMANDI BASE:

- **M** : MOLTIPLICAZIONE / DIVISIONE .
- **A** : OPERAZIONI ATOMICHE .
- **F** : FLOATING POINT SINGLE PRECISIONE .
- **D** : " " " " DOPPIA " " .
- **C** : COMPRESSED (Rende + corte le istruzioni) .
- **V** : VETTORIALE .
- **G** : GENERAL PURPOSE .

LE ESTENSIONI SONO MODULARI E SI COMBINANO LIBERAMENTE.

QUALI SONO I LIVELLI DI PRIVILEGIO DI PISC-V?

- **LIVELLO 0** : USER / APPLICATION → ESEGUONO PROGRAMMI UTENTE
- **LIVELLO 1** : SUPERVISOR (OPZIONALE) → GESTISCE S.O., MEMORIA VIRTUALE, ECC.
- **LIVELLO 2** : HYPERVISOR → GESTIONE DI MACCHINE VIRTUALI
- **LIVELLO 3** : MACHINE → ACCESSO DIRETTO ALL'HARDWARE, ECCEZIONI CRITICHE E RESET.

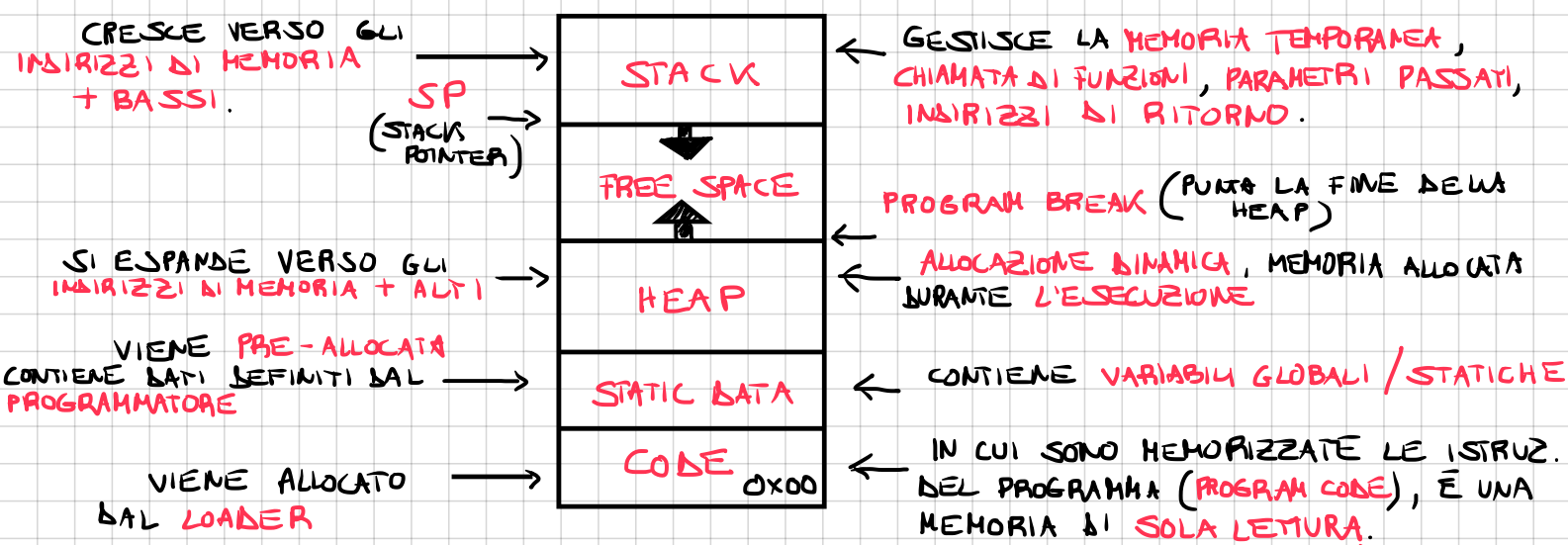
RISC-V LAVORA TIPICAMENTE IN USER E MACHINE

COME È ORGANIZZATA LA STRUTTURA DI RV32I?

VISTO CHE APPARTIENE APPARTIENE ALLA FAMIGLIA RISC ADOTTA UN MODELLO BASATO SUI REGISTRI.

LE OPERAZIONI LOGICO-ARITMETICHE VENGONO SVOLTE NEI REGISTRI, I DATI VENGONO PRELEVATI DALLA MEMORIA TRAMITE ISTRUZIONI **LOAD** E **STORE** (ARCHITETTURA **LOAD-STORE**). UTILIZZA ANCHE LO **STACK**, GESTITO DA UN REGISTRO **SP** (**STACK POINTER** → REGISTRO **X2**), CHE SERVE PER CONTENERE **DATI TEMPORANEI**, **VARIABILI LOCALI** E **INDIRIZZI DI RITORNO** QUANDO I REGISTRI NON BASTANO PIÙ O SI ENTRA IN UNA **FUNZIONE**. LA MEMORIA IN RISC-V RV32I È ORGANIZZATA IN UNA STRUTTURA CHIAMATA **MEMORY LAYOUT**.

COME È COSTITUITO IL MEMORY LAYOUT?



SECONDO QUALI FATTORI IL SISTEMA OPERATIVO ASSEGNA DIMENSIONI, SPAZIO E DISPOSIZIONE DELLE VARIE REGIONI DI MEMORIA?

- **LIMITI DEL SISTEMA OPERATIVO**: IL KERNEL IMPONE DEI LIMITI IN MODO TALE CHE UN SINGOLO PROCESSO NON CONSUMI TUTTA LA MEMORIA.
- **CONFIGURAZIONE UTENTE**: IN MOLTI SISTEMI I LIMITI POSSONO ESSERE CONFIGURATI PER POTER MODIFICARE AD ESEMPIO LA DIMENSIONE DELLO STACK PER PROCESSO (**RLIMIT_STACK** SU LINUX)
- **ARCHITETTURA E SPAZIO DI INDIRIZZAMENTO**: L'ARCHITETTURA DELLA CPU DETERMINA QUANTO SPAZIO DI INDIRIZZI VIRTUALI È DISPONIBILE E QUINDI HEAP E STACK POSSONO CRESCERE.

COSA È IL MEMORY ALLOCATOR?

IL MEMORY ALLOCATOR È UN COMPONENTE DELLA **LIBRERIA RUNTIME** CHE GESTISCE LA **MEMORIA DINAMICA** NELL' **HEAP**. QUANDO UN PROGRAMMA USA FUNZIONI COME **MALLOC**, **CALLOC**, **REALLOC**, **FREE** NON MANIPOLA DIRETTAMENTE LA MEMORIA MA ASSEGNA IL COMPITO ALL' **ALLOCATOR**.

IL SUO COMPITO È QUELLO DI **TENERE TRACCIA** DELLE ZONE DI MEMORIA **GIÀ UTILIZZATE** OPPURE **DISPONIBILI**.

SE TROVA MEMORIA LIBERA NELL' **HEAP** RESTITUISCE IL PUNTERE ALLO SPAZIO DI MEMORIA SENZA **COINVOLGERE IL SISTEMA OPERATIVO**, INVECE SE L' **HEAP** NON CONTIENE ABBASTANZA MEMORIA CHIEDE ALTRA MEMORIA AL **SISTEMA OPERATIVO**, TRAMITE UNA **SYSTEM CALL**, IL SISTEMA OPERATIVO POI DECIDE SE CONCEDERE NUOVA MEMORIA. QUANDO LA MEMORIA NON È PIÙ NECESSARIA, IL PROGRAMMA RICHAMA **FREE** E L' **ALLOCATOR** SEGNA COME DISPONIBILE QUEL BLOCCO DI MEMORIA, PER FARLO L' **ALLOCATOR** POSSIEDE **STRUTTURE INTERNE** CHE **DESCRIVONO L'HEAP** E OVVIAMENTE DEVE GESTIRE ANCHE LA **FRAMMENTAZIONE DELLA MEMORIA**.

COSA È IL MEMORY LEAK?

È UNA SITUAZIONE IN CUI LA MEMORIA ALLOCATA NELLA **HEAP** NON VIENE LIBERATA, RENDENDO QUELLA PORZIONE DI MEMORIA NON UTILIZZABILE E IRRAGGIUNGIBILE PER TUTTO IL PROGRAMMA. QUINDI IN C IL PROGRAMMATORE DEVE LIBERARE LA MANUALMENTE, IN PYTHON E JAVA ESISTE IL **GARBAGE COLLECTOR** CHE LIBERA AUTOMATICAMENTE LA MEMORIA UTILIZZATA (LEGGERO OVERHEAD RUNTIME).

COME È ORGANIZZATA LA MEMORIA IN RISCV?

LA MEMORIA È **BYTE-ADDRESSABLE**, ONERO OGNI INDIRIZZO PUNTA AD 1 BYTE, VISTO CHE GLI INDIRIZZI SONO A 32 BIT, POSSO INDIRIZZARE 2^{32} BYTE QUINDI 4GB.

RISCV ADOTTA IL **LITTLE ENDIAN**, IL BIT MENO SIGNIFICATIVO NELL' INDIRIZZO È LA MEMORIA PIÙ BASSA. (RV32I SUPPORTA ANCHE IL **BIG ENDIAN**)

INOLTRE I DATI DEVONO ESSERE ALLINEATI SECONDO LA LORO **DIMENSIONE**:

- **WORD** (4 BYTE) → INDIRIZZO MULTIPLO DI 4

- **HALFWORD** (2 BYTE) → INDIRIZZO MULTIPLO DI 2

SE NON FOSSERO ALLINEATI GENEREREBBE UN'ECCEZIONE OPPURE PIÙ CICLI DI MACCHINA.

LO SPAZIO DI MEMORIA È **FLAT MEMORY MODEL**, NON ESISTONO ZONE DI MEMORIA SEPARATE È UNA SEQUENZA **CONTINUA** DI INDIRIZZI (DA 0 A 4GB).

INOLTRE TUTTI I DISPOSITIVI (RAM, ROM, I/O) CONDIVIDONO LA **STESSA** MEMORIA, QUESTO MECCANISMO È CHIAMATO **MEMORY-MAPPED I/O (MMIO)**.

LA MEMORIA IN RISC-V SEGUE IL MODELLO **RVWMO** (RISC-V WEAK MEMORY ORDERING) OVELE LE ISTRUZIONI **NON SONO NECESSARIAMENTE ESEGUITE** NELL'ORDINE SCRITTO NEL CODICE, MA POSSONO ESSERE **RIORDATE** PER **MIGLIORARE** LE PRESTAZIONI.

QUALI TIPI DI DATO SONO IMPLEMENTATI?

RV32I TRATTI **NUMERI INTERI** CON SEGNO IN COMPLEMENTO A2, IN **NUMERI UNSIGNED** SONO SUPPORTATI TRAMITE **VARIANTI** DI ISTRUZIONI.

I **TIPI COMPLESSI** VENGONO GESTITI COME SEQUENZE DI **WORD** O **BYTE**.

QUANDO SI TRADUCE UN PROGRAMMA IN C IN **RV32I**, BISOGNA CONVERTIRE I TIPI DI DATO IN C NEI TIPI DI DATO NATIVI DI **RV32I** (I PUNTORI SONO **UNSIGNED WORD**).

QUALI ISTRUZIONI ESISTONO IN RV32I?

CI SONO UN NUMERO LIMITATO DI **40** ISTRUZIONI BASE, OGNIUNA DI QUESTE VIENE TRADOTTA DALL' **ASSEMBLER** NELLA CORRISPONDENTE **ISTRUZIONE MACCHINA**.

ESISTONO POI DELLE **PSEUDO-ISTRUZIONI** CHE NON HANNO UNA CORRISPONDENZA IN **BINARIO**, MA VENGONO RICONOSCIUTE DALL'ASSEMBLER E TRADOTTE IN **UNA O PIÙ** ISTRUZIONI.

QUAL È LA NOTAZIONE "STILE VERILOG" ADOTTATA DA LIBRO?

• RIFERIMENTO A **REGISTRO** E **MEMORIA**:

- **R[RS1]**: LEGGI / SCRIVI NEL REGISTRO **RS1**.

- **M[RS1]**: LEGGI / SCRIVI NELLA MEMORIA PUNTATA DA **RS1**.

ES: $R[RS1] = R[RS1] + M[RS2]$ # SALVO NEL REGISTRO RS1 LA SOMMA DATA DAL SUO CONTENUTO E LA MEMORIA PUNTATA DAL REGISTRO RS2.

- BIT SLICING ($i:j$):

- $R[RS1](31) \rightarrow$ IL BIT PIÙ SIGNIFICATIVO DEL CONTENUTO DEL REGISTRO RS1.
- $R[RS1](7:0) \rightarrow$ 1 BIT DAL 7 AL 0 (8 BIT) DEL " " " " .

- **CONCATENAZIONE DI BIT** $\{\}$:

$$\{R[RS1] \ (31:16), R[RS2] \ (15:0)\}$$

CONCATENA I 16 BIT PIÙ SIGNIFICATIVI DEL CONTENUTO DI RS1 CON I 16 BIT PIÙ SIGNIFICATIVI DEL CONTENUTO DI RS2.

- **DUPLICAZIONE BIT :**

24'bm[] (7) → CREA 24 COPIE DEL BIT 7.

ES: $R[R_D] = \{24' \text{ bM}[] (7), \text{M}[R[R_{S1}] + 144] (7:0)\}$

OVVERO: SALVA NEL REGISTRO R1, IL BYTE MENO SIGNIFICATIVO DEL BYTE CONTENUTO IN MEMORIA PUNTATO DA R5 + IMM E NE ESTENDE IL BIT DI SEGNO.

- CONDIZIONI INLINE :

$$R[RS_1] = (R[RS_1] \triangleleft R[RS_2])^2 \quad 1 : 0$$

COME L'OPERATORE TERNARIO IN C, SE IL CONTENUTO DI RS1 È MINORE DEL CONTENUTO IN RS2 ALLORA SALVO 1 IN RS1 SENNO' 0.

QUALI SONO I COMANDI DI TRASFERIMENTO DATI?

LOAD : LB, LH, LW, LBU, LHU.

STORE : SB, SH, SW.

LOAD :

- **LB** (LOAD BYTE) : CARICA NELLA MEMORIA UN VALORE DI 8 BIT A CUI VIENE ESTESO IL BIT DI SEGNO FINO ALLA LUNGHEZZA DEL REGISTRO (32 BIT) PRIMA DI SCRIVERLO IN RD.

$$LB \quad RD, \text{ OFFSET } (RS1) \rightarrow R[RD] = \{ 24'bM7, M[R[RS1]+imm](7:0) \}$$

- **LH** (LOAD HALF WORD): UGUALE A LB MA CARICA UN VALORE DI 16 BIT E ESTENDE IL SEGNO FINO A 32 PRIMA DI SALVARLO IN R3.

$$\text{LH } R_D, \text{ OFFSET } (RS_1) \rightarrow R[R_D] = \{16 \cdot \text{BM}[] (15), M[R[RS_1] + 144] (15:0)\}$$

- **LW** (LOAD WORD) : CARICA DALLA MEMORIA UN VALORE DI 32 BIT E LO METTE IN RD

$$LW\ RD, OFFSET(RS1) \rightarrow R[RD] = M[R[RS1] + IMM]$$
- **LBU** (LOAD BYTE UNSIGNED) : UGUALE A **LB** MA ESTENDE CON ZERI.

$$LBU\ RD, OFFSET(RS1) \rightarrow R[RD] = \{24' b0, M[R[RS1] + IMM](7:0)\}$$
- **LHU** (LOAD HALF WORD UNSIGNED)

$$LHU\ RD, OFFSET(RS1) \rightarrow R[RD] = \{16' b0, M[R[RS1] + IMM](15:0)\}$$

STORE:

- **SB** (STORE BYTE) : MEMORIZZA IN MEMORIA GLI 8 BIT MENO SIGNIFICATIVI DEL CONTENUTO DI RS2.

$$SB\ RS2, OFFSET(RS1) \rightarrow M[R[RS1] + IMM](7:0) = R[RS2](7:0)$$
- **SH** (STORE HALF WORD) : MEMORIZZA IN MEMORIA I 16 BIT MENO SIGNIFICATIVI DEL CONTENUTO DI RS2.

$$SH\ RS2, OFFSET(RS1) \rightarrow M[R[RS1] + IMM](15:0) = R[RS2](15:0)$$
- **SW** (STORE WORD) : MEMORIZZA IN MEMORIA I 32 BIT DEL REGISTRO DE CONTENUTO DI RS2.

$$SW\ RS2, OFFSET(RS1) \rightarrow M[R[RS1] + IMM](31:0) = R[RS2](31:0)$$

QUALI SONO LE ISTRUZIONI ARITMETICHE?

LE OPERAZIONI ARITMETICHE USANO COME OPERANDI REGISTRI O IMMEDIATI:

- **ADD** → MEMORIZZA NEL REGISTRO RD LA SOMMA TRA RS1 E RS2.

$$ADD\ RD, RS1, RS2 \rightarrow R[RD] = R[RS1] + R[RS2]$$
- **SUB** → MEMORIZZA NEL REGISTRO RD LA DIFFERENZA TRA RS1 E RS2.

$$SUB\ RD, RS1, RS2 \rightarrow R[RD] = R[RS1] - R[RS2]$$
- **ADDI** → MEMORIZZA NEL REGISTRO RD LA SOMMA TRA RS1 E IMM.

$$ADDI\ RD, RS1, IMM \rightarrow R[RD] = R[RS1] + IMM$$

SUBI NON ESISTE PERCHÉ BASTA USARE **ADDI** CON **IMM NEGATIVO**. MAX
 GLI IMM SONO **COSTANTI SIGNED A 12 BIT** ($ADDI\ RD, RS1, 0xFFF$)

QUALI SONO LE ISTRUZIONI LOGICHE?

HANNO COME OPERANDI REGISTRI O IMMEDIATI.

- **AND** → ESEGUE LA AND BIT A BIT TRA RS1 E RS2 E MEMORIZZA IN RD.

$$\text{AND } RD, RS1, RS2 \rightarrow R[RD] = R[RS1] \& R[RS2]$$

- **OR** → ESEGUE LA OR BIT A BIT TRA RS1 E RS2 E MEMORIZZA IN RD.

$$\text{OR } RD, RS1, RS2 \rightarrow R[RD] = R[RS1] | R[RS2]$$

- **XOR** → ESEGUE LA XOR BIT A BIT TRA RS1 E RS2 E MEMORIZZA IN RD.

$$\text{XOR } RD, RS1, RS2 \rightarrow R[RD] = R[RS1] \wedge R[RS2]$$

↳ SE I BIT SONO DIVERSI È 1, SE SONO UGUALI 0.

- **ANDI, ORI, XORI** → ESEGUONO OPERAZIONI BIT A BIT TRA RS1 E IMM.

$$\text{ANDI } RS1, IMM \rightarrow R[RD] = R[RS1] \& IMM$$

$$\text{ORI } RS1, IMM \rightarrow R[RD] = R[RS1] | IMM$$

$$\text{XORI } RS1, IMM \rightarrow R[RD] = R[RS1] \wedge IMM$$

} IMM È SIGNED A 12 BIT
[-2048, 2047]

ANDI → MASCHERA FLAG E LEGGE CAMPI DI REGISTRI

ORI → IMPOSTA BIT SPECIFICI, ABILITA FLAG / PERIFERICHE

XORI → INVERTE I BIT, CHECKSUM, CRITTOGRAFIA

QUALI SONO LE ISTRUZIONI DI SCORRIAMENTO (SHIFT)?

SONO 6: **SLL, SLLI, SRL, SRLI, SRA, SRAI**.

- **SLL & SLLI**: SHIFT LOGICO A SINISTRA, UTILIZZATE PER MOLTIPLICARE CON POTENZE DI 2 (AGGIUNGENDO 0 COME BIT MENO SIGNIFICATIVO)

$$\text{SLL } RD, RS1, RS2 \rightarrow R[RD] = R[RS1] \ll R[RS2]$$

$$\text{SLLI } RD, RS1, IMM \rightarrow R[RD] = R[RS1] \ll IMM \rightarrow 5 \text{ BIT UNSIGNED } [0, 31]$$

- **SRL & SRLI**: SHIFT LOGICO A DESTRA, UTILIZZATO PER DIVIDERE CON POTENZE DI 2 (AGGIUNGE 0 COME BIT PIÙ SIGNIFICATIVO QUINDI NON HA SENSO PER I NEGATIVI).

$$\text{SRL } RD, RS1, RS2 \rightarrow R[RD] = R[RS1] \gg R[RS2]$$

$$\text{SRLI } RD, RS1, IMM \rightarrow R[RD] = R[RS1] \gg IMM \rightarrow 5 \text{ BIT UNSIGNED } [0, 31]$$

- **SRA & SRAI**: SHIFT ARITMETICO A DESTRA, UGUALE ALLO SHIFT LOGICO DESTRO MA NEI BIT PIÙ SIGNIFICATIVI VIENE REPLICATO IL SEGNO, QUINDI HA SIGNIFICATO ARITMETICO.

QUALI SONO LE ISTRUZIONI DI CONTROLLO DEL FLUSSO?

IN RISCV OGNI ISTRUZIONE OCCUPA 4 BYTE, QUINDI DOPO AVER ESEGUITO L'ISTRUZIONE ALL'INDIRIZZO 0X8000, IL PROCESSORE PRELEVA LA SUCC. DA 0X8004.

QUESTE ISTRUZIONI MODIFICANO QUESTA LOGICA.

ESISTONO 2 TIPI DI SALTI:

- **CONDIZIONATI** o **INCONDIZIONATI**: IL SALTO DIPENDE O NO DA UNA **CONDIZIONE**.
- **DIRETTO** o **INDIRETTO**: L'INDIRIZZO A CUI SALTARE SIA **ESPLICITAMENTE INDICATO** o **CONTENUTO IN UN REGISTRO**.

SALTO DIRETTO → ES: BEQ A0, A1, L (SALTA ALL'ETICHETTA L)
INCORPORATA NELL'ISTRUZIONE

SALTO INDIRETTO → ES: JALR RS1, OFFSET (SALTA ALL'INDIRIZZO CONTENUTO
IN RS1 + UN OFFSET)
12 BIT

CONDIZIONALI: BEQ, BNE, BLT, BGE, BLTU, BGEU.

- **BEQ**: SE I 2 REGISTRI CONTENGONO LO STESSO VALORE AVVIENE IL SALTO

BEQ RS1, RS2, OFFSET ⇒ IF (RS1 == RS2) PC += OFFSET ELSE PC += 4.

INTERO **SIGNED** (IL SALTO PUÒ AVVENIRE INDIETRO O AVANTI)

↳ 12 BIT CHE SAREBBE $[-2^{11}, 2^{11}-1]$ BYTE

MA VISTO CHE IL PRIMO BIT È SEMPRE ZERO È COME SE FOSSE 13 BIT (VISTO CHE L'ULTIMO BIT NON LO METTIAMO): $[-2^{12}, 2^{12}-1]$ BYTE,

QUINDI SE LE ISTRUZIONI SONO COMPRESSE A 16 BIT (ESTENSIONE C) -2048, +2047 HALF WORD, SE STANO IN RV32I (4 BYTE DI ISTRUZIONE) POSSIAMO SALTARE CON UN RANGE DI $[-1024, +1023]$ ISTRUZIONI.

IMPORTANTE

- **BNE**: UGUALE MA CONTROLLA SE SONO DIVERSI.
- **BLT**: CONTROLLA SE RS1 È MINORE DI RS2.
- **BGE**: CONTROLLA SE RS1 È MAGGIORE O UGUALE DI RS2.
- **BLTU** E **BGEU** SVOLGONO LA STESSA FUNZIONE MA INTERPRETANO I VALORI **SENZA SEGNO** COME QUELLI CON SEGNO.

X5 = -1, X6 = 1

BLT X5, X6, LABEL ⇒ **VERO** $-1 < 1$

BLTU X5, X6, LABEL ⇒ **FALSO** $4.294.967.295 < 1$

INCONDIZIONALI : JAL E JALR.

- **JAL** : ESEGUE SEMPRE UN SALTO A UN NUOVO INDIRIZZO ($PC + \text{OFFSET}$) DURANTE L'OPERAZIONE IL PROCESSORE SI SALVA **PRIMA** $PC + 4$ DENTRO **RD**, IN MODO TALE DA TORNARE AL PUNTO DI PARTENZA.

$JAL\ RD, \text{OFFSET} \Rightarrow RD = PC + 4 \quad PC = PC + \text{OFFSET}$

20 BIT **SIGNED** $\rightarrow [-2^{20}, +2^{20}-1]$ PERCHÉ IL BIT MENO SIGNIFICATIVO È SEMPRE ZERO

PER ISTRUZIONI COMPRESSE A 2 BYTE SONO $[-2^{19}, +2^{19}-1]$ HALF WORDS.

IN RV32I CON ISTRUZIONI 4 BYTE ABBIAMO $[-2^{18}, +2^{18}-1]$ ISTRUZIONI

- **JALR** : ESEGUE UN SALTO **INCONDIZIONATO** E **INDIRETTO**, L'INDIRIZZO VIENE CALCOLATO SOMMANDO IL VALORE NEL REGISTRO $RS1 + \text{OFFSET}$ IMMEDIATO CON SEGNO, SALVANDO $PC + 4$ IN **RD**.
(TIPICAMENTE USATO PER **CHIAMATE** O **RITORNI** DI FUNZIONI)

$JALR\ RD, RS1, \text{OFFSET} \Rightarrow RD = PC + 4$
 $PC = (RS1 + \text{OFFSET}) \& \sim 1$ NOT 1

VISTO CHE LE ISTRUZIONI SONO ALLINEATE ALLA HALF WORD (CON ESTENSIONE C) DOBBIAMO AZZERARE IL BIT MENO SIGNIFICATIVO

12 BIT **SIGNED**

HA È IN UNITÀ DI **BYTE**, OVERO NON

METTE PER SCOMATO CHE IL BIT MENO SIGNIFICATIVO SIA ZERO PERCHÉ $RS1$ POTREBBE CONTENERE QUALSIASI INDIRIZZO

ES : $(RS1 + \text{OFFSET}) \& \sim 1 \rightarrow$

$\begin{array}{r} 1110101 \\ 1111110 \\ \hline 1110100 \end{array}$

ALLINEATO
AS HALF WORD

$\rightarrow$ RANGE $[-2^{11}, +2^{11}-1]$ BYTE

$[-2^{10}, +2^{10}-1]$ HALF WORD (2 BYTE)

$[-2^9, +2^9-1]$ ISTRUZIONI (4 BYTE)

IMPORTANTE!!

QUINDI I **SALTI CONDIZIONALI** (BEQ, BLT, BGE, ...) VENGONO USATI PER **IF**,

CICLI, **SALTI LOCALI**, QUINDI DEVONO ARRIVARE IL + LONTANO POSSIBILE, SANNO

CHE LE ISTRUZIONI SONO ALLINEATE AD HALF WORD QUINDI SCARTANO IL BIT MENO SIGNIFICATIVO (È SEMPRE ZERO) **RADDOPPIANDO IL RANGE**, STESSA COSA FA

JAL PERCHÉ USA IL PC CHE È GIÀ **ALLINEATO**.

INVECE **JALR** OPERA SU INDIRIZZI **BYTE-LEVEL** PERCHÉ UTILIZZA IL CONTENUTO

DI UN REGISTRO COME INDIRIZZO E POICHÉ LA MEMORIA È **BYTE ADDRESSABLE**

L'OFFSET DEVE PERMETTERE DI FARE CALCOLI PRECISI SU PUNTATORI E STRUTTURE.

QUALI SONO LE ISTRUZIONI DI CONFRONTO?

Sono: **SLT**, **SLTI**, **SLTU**, **SLTIU**

- **SLT** (SET LESS THEN) E **SLTU** (SET LESS THEN UNSIGNED) CONFRONTANO IL CONTENUTO DI **RS1** E **RS2** E IMPOSTANO **RD** A **1** SE IL PRIMO È STRETT. MINORE DEL SECONDO ALTRIMENTI LO IMPOSTA A **0**.

SLT INTERPRETA GLI OPERANDI CON SEGNO, **SLTU** INVECE SENZA SEGNO:

$$\text{SLT } RD, RS1, RS2 \Rightarrow R[RD] = R[RS1] <_S R[RS2]$$

$$\text{SLTU } RD, RS1, RS2 \Rightarrow R[RD] = R[RS1] <_U R[RS2]$$

- **SLTI** E **SLTIU** FANNO LA STESSA COSA MA CONFRONTANO **RS1** E IMM.

$$\text{SLTI } RD, RS1, IMM \Rightarrow R[RD] = R[RS1] <_S IMM$$

$$\text{SLTIU } RD, RS1, IMM \Rightarrow R[RD] = R[RS1] <_U IMM$$

QUALI SONO LE ISTRUZIONI DI INDIRIZZAMENTO?

SERVONO PER GESTIRE IMMEDIATI, CARICARE O MODIFICARE REGISTRI O IL PC E SONO FONDAMENTALI PER **COSTRUIRE** INDIRIZZI DI MEMORIA, ACCEDERE A DATI COSTANTI O CALCOLARE **SALTI A LUNGO RAGGIO**.

JAL DISPONE DI 20 BIT MENTRE LE ISTRUZIONI CONDIZIONALI (**BEQ**, **BNE**, ...) HANNO SOLI 12 BIT.

- **LUI** (LOAD UPPER IMMEDIATE): CARICA UN VALORE IMMEDIATO NEI **20 BIT** + **ALTI** DEL REGISTRO **RD**, AZZERANDO I 12 BIT INFERIORI

$$\text{LUI } RD, IMM \Rightarrow R[RD] = IMM \ll 12$$

UTILIZZO PRATICO (CARICARE COSTANTI + GRANDI DI 20 BIT)

$$\text{LUI } X5, 0x12345 \quad \# \quad X5 = 0x12345000$$

$$\text{ADDI } X5, 0x678 \quad \# \quad X5 = 0x12345678$$

OPPURE PER COSTRUIRE INDIRIZZI DI MEMORIA RELATIVI + GRANDI DI 20 BIT.

ES: `UINT32_T ARRAY[4] = {10, 20, 30, 40};` ALLOCATO ALL'INDIRIZZO **0x20001000**

SUPPONIAMO DI VOLER CARICARE `ARRAY[2]`:

$$\begin{aligned} 1) \quad & \text{LUI } X10, 0x20001 \quad \# \quad 20 \text{ BIT} + \text{SIGNIFICATIVI} \quad X10 = 0x20001000 \\ & \text{ADDI } X10, X10, 8 \quad \# \quad X10 = 0x20001008 \quad 2 \text{ POS IN AVANTI} \\ & \text{LW } X5, 0(X10) \quad \# \quad \text{LEGGE IL VALORE IN } X5. \end{aligned}$$

2) LUI X10, 0x20001 # X10 = 0x20001000

LW X5, 8(X10) # LEGGE ARRAY[2] DIRETTAMENTE

- **AUIPC** (ADD UPPER IMMEDIATE TO PC) : UGUALE A LUI MA L'IMMEDIATO SHIFTATO VIENE SOMMATO AL PROGRAM COUNTER, PER COSTRUIRE INDIRIZZI DI MEMORIA RELATIVI AL PC, UTILE PER SALT, ACCESSO AI DATI O STRUTTURE IN MEMORIA.

$AUIPC\ Rn, IMM \Rightarrow R[Rn] = PC + (IMM \ll 12)$

VISTO CHE JAL CI PERMETTE DI SALTARE DI $\pm 1\text{MIB}$ CIRCA $(-2^{20}, +2^{20}-1)$, SE VOLESSIMO SALTARE A UN'ISTRUZIONE CHE SI TROVA A $+4\text{MIB}$ DAL PC, NON POTREMMO FARLO, ALLORA:

SOLUZIONE: AUIPC + JALR

AUIPC X1, 0x400 # X1 = PC + $\underbrace{0x400000}_{4\text{MIB}}$

JALR X1, 0(X1) # SALTA ALL'INDIRIZZO X1 CALCOLATO PRIMA E SALVO
IN X1 IL PC+4 (PER RITORNARE)

INVECE QUANDO HO UN SALTO CONDIZIONATO CON UN SALTO TROPPO LUNGO:

BEQ X1, X2, TARGET # TARGET È TROPPO LONTANO

METODO 1: INVERTO LA CONDIZIONE

BNE X1, X2, SKIP

SE HO BISOGNO DI CALCOLARE LA PARTE ALTA

LUI E0, 0x12345 # PARTE ALTA

ADDI E0, E0, 0x678 # PARTE BASSA OPPURE DIRETTAM. JALR X0, 0x678(E0)

JALR X0, 0(E0) # SALTO E "CESTINO" PC+4 IN X0 TANTO NON SERVE TORNARE INDIETRO

SKIP:

METODO 2: SENZA INVERTIRE

BEQ X1, X2, VERO

SKIP:

JAL X0, SKIP

VERO:

LUI E0, PARTE-ALTA

JALR E0, PARTE-BASSA(E0)

QUALI SONO LE ISTRUZIONI DI SINCRONIZZAZIONE DELLA MEMORIA?

SERVONO PER:

- INTERAZIONE CON L'AMBIENTE DI ESECUZIONE (OS, TRAP, DEBUGGER).
- ORDINE E VISIBILITÀ DELLE OPERAZIONI DI MEMORIA E I/O TRA PROCESSI, CORE O DISPOSITIVI.
- CONTROLLO E DIAGNOSTICA DELL'ESECUZIONE

• **FENCE**: ORDINARE GLI ACCESSI A DISPOSITIVI I/O E ALLA MEMORIA

PRED → INDICA QUALI OPERAZIONI ESEGUIRE PRIMA DI FENCE.

SUCC → INDICA QUALI OPERAZIONI ESEGUIRE DOPO LA FENCE.

FENCE PRED, SUCC → FENCE (PRED, SUCC)

QUINDI GARANTISCE CHE ALCUNI ACCESSI AVVENGONO PRIMA DI ALTRI, COSÌ GLI ALTRI HART O DISPOSITIVI ESTERNI LO VEDANO NELLO STESSO ORDINE.

↳ THREAD HARDWARE

FENCE RW, W → COMPORTA CHE TUTTE LE SCRITTURE E LETTURE PRECEDA VENGANO COMPLETATI PRIMA DELLA FENCE.

↳ COMPORTA CHE TUTTE LE SCRITTURE SUCCESSIVE VENGANO COMPLETATE DOPO FENCE.

ES: UN PROCESSORE RISC-V CON 2 HART, UNO DEVE SCRIVERE L'ALTRO LEGGERE, L'ORDINE POTREBBE ESSERE INVERTITO DAL PROCESSORE PER OTTIMIZZARE:

SW t0, 0(A0) # SCRIVO

LW t1, 4(A0) # LEGGO

⇒

SW t0, 0(A0)

FENCE W, R # FORZO LA SCRITTURA PRIMA DELLA LETTURA.

LW t1, 4(A0)

- **ECALL**: EFFETTUA UNA RICHIESTA ALL'AMBIENTE DI ESECUZIONE DI SUPPORTO, ESEGUE IN BASE SE VIENE ESEGUITA IN U-MODE, S-MODE O M-MODE UNA ECCEZIONE DI CHIAMATA ALL'AMBIENTE.

ECALL → RAISE EXCEPTION (ENVIRONMENTCALL)

- **EBREAK**: USATO DAL DEBUGGER PER FAR SÌ CHE IL CONTROLLO VENGA TRASFERITO DI NUOVO ALL'AMBIENTE DI DEBUG.

EBREAK → RAISE EXCEPTION (BREAK POINT)

↳ FERMA IL PROGRAMMA E PASSA IL CONTROLLO A CHI LO STA ESEGUENDO (REVISIONE).

QUALI SONO LE PSEUDOISTRUZIONI?

SONO DELLE SCORciatoIE CHE L'ASSEMBLATORE TRADUCE IN 1 o + ISTRUZIONI REALI, CHE DIPENDO DALL'ASSEMBLATORE.

- **LA** $RD, \underbrace{SYMBOL}_{32 \text{ BIT}}$ $\rightarrow$ **LUI + ADDI** PER CARICARE DIRETTAMENTE ETICHETTE DA 32 BIT.
(SENZA I 20 + SIGNIFICATIVI CON **LUI** E I 12 - SIGNIFICATIVI CON **ADDI**)
- **LI** RD, IMM $\rightarrow$ **ADDI** O **LUI + ADDI** PER CARICARE IMMEDIATI FINO A 32 BIT.
- **CALL** **LABEL** $\rightarrow$ **AUIPC RA, ... + JALR RA, ...** SERVE PER FARE CHIAMATE DI FUNZIONI
PRATICAMENTE SE DOBBIAMO SALVARE A: **0X12345678**

$$\left. \begin{array}{l} \text{AUIPC RA, 0X12345} \\ \text{JALR RA, 0X678 (RA)} \end{array} \right\} \Rightarrow \text{ECALL LABEL} \rightarrow \text{ETICHETTA A QUELL'INDIRIZZO}$$

QUAL È IL FORMATO DELLE ISTRUZIONI?

	31	25 24	20 19	15 14	12 11	7 6	0
R	FUNZ 7		RS2	RS1	FUNZ 3	RD	CODOP
I	IMM [11:0]			RS1	FUNZ 3	RD	CODOP
S	IMM [11:5]		RS2	RS1	FUNZ 3	IMM [4:0]	CODOP
SB	IMM [12 10:5]		RS2	RS1	FUNZ 3	IMM [4:1 11]	CODOP
U	IMM [31:12]					RD	CODOP
UJ	IMM [20 10:1 11 19:12]					RD	CODOP

CODOP $\rightarrow$ 7 BIT

FUNZ 3 / FUNZ 7 $\rightarrow$ ALCUNE ISTRUZ. HANNO LO STESSO CODOP, QUINDI QUESTI CAMPI SERVONO PER DISTINGUERLI
 $\downarrow$ $\downarrow$
 3 BIT 7 BIT

REGISTRI $\rightarrow$ 5 BIT

IMMEDIATE / OFFSET $\rightarrow$ A SECONDA DEL FORMATO 12 / 20 BIT

QUALI SONO I TIPI DI ISTRUZIONI?

- **R-TYPE** : ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT E SLTU (10)
- **I-TYPE** : ADDI, ANDI, ORI, XORI, SLLI, SRAI, SRLI, SLTI, SLTIU, LB, LH, LW, LBU, LHU, JALR, EBREAK, ECALL, FENCE (18)
- **S-TYPE** : SB, SH, SW (3).
- **B-TYPE** : BEQ, BGE, BGEU, BLT, BLTU, BNE (6).
- **U-TYPE** : LUI, AUIPC (2).
- **J-TYPE** : JAL (1).

QUALI SONO LE MODALITÀ DI INDIRIZZAMENTO UTILIZZABILI?

- **TRAMITE REGISTRO** : L'OPERANDO È NEL REGISTRO (R-TYPE).

$RD \leftarrow RS1 \text{ OP } RS2$

- **IMMEDIATO** : L'OPERANDO È CODIFICATO NELLA ISTRUZIONE (I-TYPE).

$RD \leftarrow RS1 \text{ OP } IMM$

- **BASE - INDICE** : L'OPERANDO SI TROVA IN MEMORIA, ALL'INDIRIZZO CALCOLATO COME **REGISTRO + OFFSET**. (S-TYPE E I-TYPE LOAD/STORE)

$LW \ x5, 8(x1) \rightarrow \text{LEGGE } x1 + 8$

$SW \ x2, 12(x3) \rightarrow \text{SCRIVE IN } x3 + 12$

- **INDICIZZATO** : PER SALTI CONDIZIONATI O NON. L'INDIRIZZO DESTINAZIONE È CALCOLATO $PC + \text{OFFSET}$. (B-TYPE, J-TYPE)

$BEQ \ x1, x2, LABEL$

$JAL \ x5, LABEL$

QUALI SONO I MECCANISMI PER LA VARIAZIONE NEL FLUSSO DI CONTROLLO?

- **BRANCH CONDIZIONATO** : BEQ, BNE, BLT, BGE.

- **JUMP INCONDIZIONATO** : JAL.

- **JUMP TRAMITE REGISTRO** : AUIPC, JALR, ECALL.

- **PASSA IL CONTROLLO AL SISTEMA OPERATIVO O TRAP HANDLER** : EBREAK.